FraudGuard: Fraud and Phishing Detection for E-Banking (Project Report) - Author: Yashitha Pobbareddy

Executive summary

FraudGuard is a working prototype that scores card transactions for fraud and web links for phishing. On simulated data it caught about 80% of fraud, and 88% of the alerts it raised were correct.

The system combines a Random Forest model, a Flask API with a dashboard, and an automated "red team" that attacks the model with simulated fraud to find where it fails. A demo is hosted on a free Render instance link: instance link. A production design on AWS is proposed in section 3. It is a plan and has not been built.

- **Detection quality:** precision 88.4%, recall 79.6%, F1 0.838 and PR-AUC 0.889 on a held out, time ordered test set of **25,703 transactions (392 fraud**). A plain model without customer history features scored F1 0.688.
- **Engineering:** rewriting the feature step to work on whole tables cut its run time by about 98% in local tests. A test proves it gives the same numbers as the one at a time version used by the API.
- **Weak spot:** the red team found that fraud which imitates the customer is mostly missed. The deployed model caught about 51% of held out simulated attacks (section 7).
- **Main caveat:** all data is simulated. The results show that the method works. They do not predict performance on real bank traffic.

1. Problem and objectives

Online banking fraud is rare but costly. A stolen card is often drained within minutes, so detection has to be fast and must avoid flagging genuine customers.

This project asks one question: can a system decide, at the moment of a transaction, whether it looks like fraud for that particular customer? The objectives were:

1. Score each transaction in real time and give a plain-English reason for every flag.
2. Detects phishing links that trick customers into giving away their login details.
3. Measure quality honestly, using metrics that suit a problem where only about 1.5% of cases are fraud.
4. Test the system against deliberate evasion, not only typical fraud.
5. Offer everything through a secure API and a dashboard that a security analyst can use.

2. Implemented system (the prototype)

The prototype is a single Python service. It loads two trained models at start-up and keeps each customer's recent history in memory.

Component	File	What it does
Data generator	`data_gen.py`	Creates about 103,000 simulated transactions for 2,500 customers, with fraud (1.5%) arriving in short bursts
Feature builder	`features.py`	Turns a transaction into 10 signals, such as spend versus the customer's average, transactions in the last hour and country mismatch. A fast table wide version is used for training and a one at a time version for the API
Training	`train.py`	Trains a baseline and a tuned Random Forest, splitting by time (first 75% to train, last 25% to test)
Phishing model	`url_features.py`	Reads 16 signals from the link text, such as length, raw IP address, risky endings and suspicious words, and classifies the link
API and dashboard	`app.py`	Flask service with API key authentication, batch scoring, reason codes and an alert feed
Integrations	`integrations.py`	Optional Google Safe Browsing lookup and alert webhook, both of which fail safely if unavailable
Red team	`red_team.py`, `redteam/`	Simulated attackers that look for blind spots and test proposed fixes
Tests	`tests/`	28 automated tests (section 5)

How one transaction is scored

1. The API checks the API key and validates the request.
2. It looks up the customer's history and works out the 10 features.
3. The Random Forest returns a risk score between 0 and 1.
4. The score becomes LOW, MEDIUM or HIGH, using the threshold chosen during training (about 0.42).
5. HIGH results get plain-English reasons and go to the alert feed and the optional webhook.

Deployment today: the prototype runs locally with one command. A demo is also hosted on a free Render instance and deployed from Git link: GitHub . The free tier sleeps when idle, so the first request after a pause can take up to about a minute.

3. Proposed AWS production architecture

The proposed design moves each prototype component onto a managed AWS service so the system can scale, survive failures and be updated safely. This is a design only. None of it is deployed.

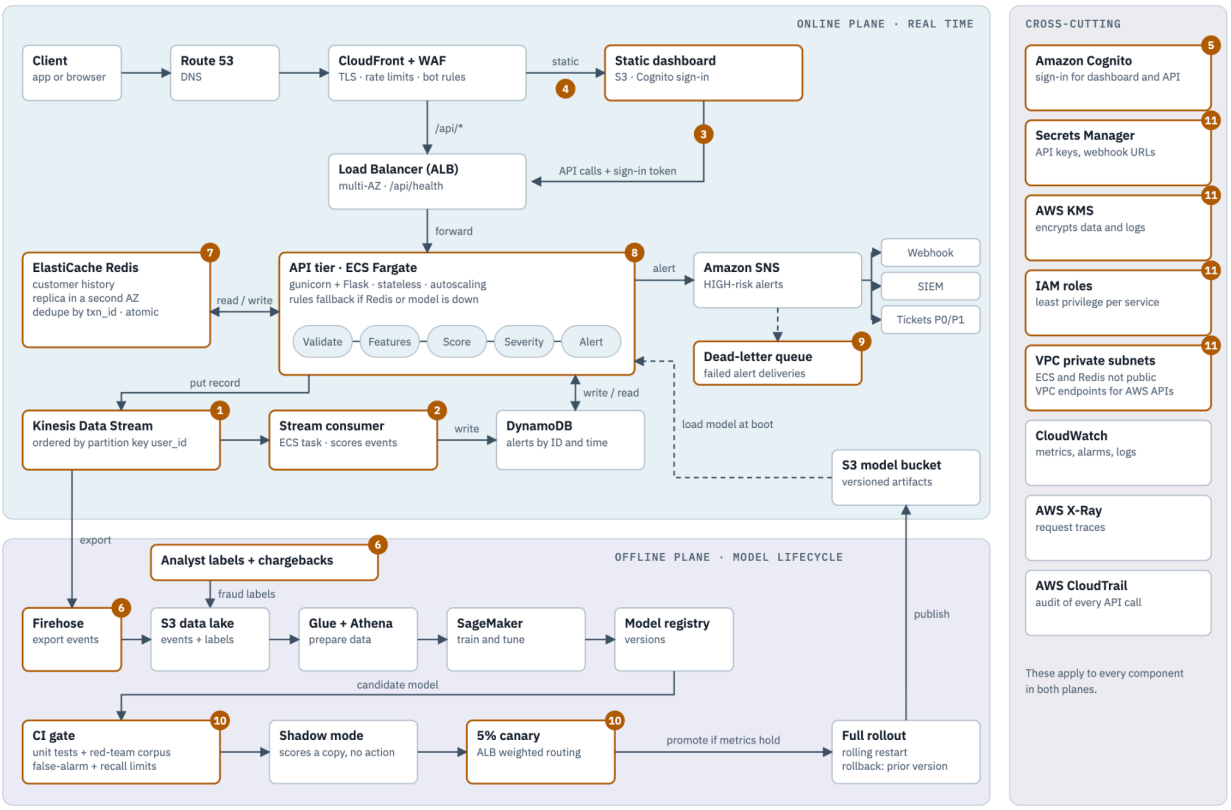


Figure 1. Proposed AWS architecture: request path (top and centre), streaming path (left) and offline model pipeline (bottom).

Reading the figure. The figure is a simplified view. In a build, the API tier (not the load balancer) would publish events to Kinesis, and the dashboard would read alerts from DynamoDB through a small read-only API.

Request path

1. A client calls over HTTPS. Route 53 sends the request to CloudFront with AWS WAF, which handles TLS, rate limits and bot rules.
2. An Application Load Balancer spread across several Availability Zones checks `/api/health` and routes traffic to healthy containers.
3. The API tier runs on ECS Fargate (gunicorn and Flask). It is stateless and scales on CPU and requests per second. Each request is validated, enriched with the customer's history, scored, given a severity and, if needed, turned into an alert.
4. Customer history sits in ElastiCache for Redis, so every container shares it and it survives restarts.
5. Alerts go to Amazon SNS, which sends them on to a webhook, a security monitoring system (SIEM) and ticketing. HIGH maps to priority P0 and MEDIUM to P1.

Streaming path. For high volume, events carry the customer ID as the partition key into Kinesis. A consumer (ECS or Lambda) scores them and writes alerts to DynamoDB, which the dashboard reads.

Partitioning by customer keeps each customer's transactions in order, which the history features need. The dashboard becomes a static site on S3 and CloudFront.

Model path (offline plane). Data lands in an S3 data lake, is prepared with Glue and Athena, and is trained and tuned in a SageMaker pipeline. The result goes into a model registry, then through a release gate (unit tests plus the red-team regression corpus), shadow mode, a 5% canary and full rollout. Rolling back means pointing to the previous version. The API loads the versioned model from S3 at start-up.

Operations. CloudWatch metrics and alarms, X-Ray traces, structured and redacted logs encrypted with KMS, and CloudTrail for audit.

How the prototype maps to AWS

Prototype today	AWS service	Reason for the change
Flask and gunicorn on Render	ECS Fargate behind an Application Load Balancer	Autoscaling, multi-zone resilience and no servers to manage
Customer history in process memory	ElastiCache for Redis	Shared by all containers and kept across restarts
Last 100 alerts in memory	DynamoDB	Durable alert storage and history
Background webhook thread	Amazon SNS	Reliable fan-out with retries to webhook, SIEM and tickets
Model files in the repository	Versioned S3 bucket and model registry	Version control and one step rollback
`train.py` run by hand	Glue, Athena and a SageMaker pipeline	Repeatable training on larger data
Red-team tests run with pytest	Release gate before rollout	Blocks a model that gets worse against known attacks
`/api/health` endpoint	Load balancer health check	Unhealthy containers are replaced automatically
Dashboard served by Flask	S3 and CloudFront static site	Cheap, fast, and takes load off the API
Console logs	CloudWatch, X-Ray and CloudTrail	Monitoring, tracing and audit

Where the same design can be used

The pipeline (event in, history lookup, score, severity, alert) fits other systems. Only the feature calculation changes.

Use	How it connects	Output
Card payments	Real-time call before a payment is approved	Allow, ask for extra verification, or block

Use	How it connects	Output
Login and activity logs	Stream consumer reading events as they arrive	Alerts within seconds
Email and web links	Plug-in or gateway extension	Quarantine or rewrite the link
A database or shared code package used by customers	Access events scored against each caller's usual behaviour	P0 and P1 risks on a dashboard

The prototype already produces risk levels and reasons. The P0 and P1 routing and the extra feature calculations for other event types would be new work.

Security and logging

- **Log decisions, not data.** Logs should hold the transaction ID, a hashed customer ID, score, level, reason codes, model version and response time. They should never hold card numbers, personal details or full links with query strings. The prototype's alert line currently logs a raw customer ID, which production would hash.
- **Protect the logs themselves.** Production logs are sensitive. They should sit in a separate account and be readable only through audited, multi-factor roles, with time limited emergency access.
- **Keep card data out of scope.** The scoring service should receive tokens and derived features, never card numbers. This reduces the compliance burden (for example PCI DSS).

Improvements to make before building

- **Authentication.** The prototype uses one shared API key. Production needs a real identity layer, such as Cognito or signed requests. A static dashboard must never contain a secret key.
- **One copy of the feature code.** Use the same feature code in the training pipeline and the API, so the model is never trained on numbers computed differently from the ones it sees live.
- **Duplicate events.** Streams can deliver an event twice. The history update should use the transaction ID so a repeat is not counted twice.
- **Redis resilience.** Run Redis with replicas across zones so it is not a single point of failure.
- **Cost.** Fargate, Redis and Kinesis have fixed monthly costs. Estimate them, and run a load test, before committing.

4. Model choice: why a Random Forest

A Random Forest was chosen because it suits this kind of data and is easy to explain. This section gives the reasoning. It is not a measured comparison against other models.

Why it fits this problem

- **Fraud signals interact.** A large amount matters far more at night, on a new device, in a foreign country. Trees capture these combinations directly.

- **More stable than one tree.** A single tree is easily thrown off by noise. Averaging 150 trees smooths that out.
- **Little preparation.** Skewed amounts and mixed feature types needed no scaling or special handling.
- **Explainable.** Feature importance and simple rules support the plain-English reason shown with each alert.
- **Practical.** It is mature in scikit-learn, easy to save and load, and scores a transaction in milliseconds.

How it was tuned. A random search tried 8 combinations of settings with 3-fold cross validation inside the training period, scored on PR-AUC. The chosen model uses 150 trees with at least 2 samples per leaf.

What to test next. Gradient boosting is the most likely rival and should be tuned and compared on the same split. An unsupervised model such as Isolation Forest could add a second layer for fraud types nobody has labelled yet. Deep learning was not tried: with about 100,000 rows and 10 numeric features it rarely beats tree models and is harder to explain.

5. Testing and validation approach

The system is tested at two levels: whether the code does what it should, and whether the model is any good. All 28 automated tests run with one command (`python -m pytest -q tests`). Twenty one pass and 7 are expected failures that record known blind spots.

Code correctness

- **Feature equivalence.** The fast training version and the one at a time API version of the features must give identical numbers. If they differed, the model would be trained on one thing and scored on another.
- **API behaviour.** A missing key returns 401, bad input returns 400, an empty or oversized batch is refused, an ordinary grocery purchase scores LOW, four fast gift-card purchases from abroad are flagged with reasons, a phishing style link is flagged and a real PayPal link is not.
- **Red-team logic.** The same random seed gives the same attack, “mimic” attacks copy the customer’s habits, card testing starts with tiny charges, a frozen account stops further losses, and mutated strategies stay in realistic ranges.
- **Regression replay.** Attacks found by the red team are saved and replayed. A scenario the model catches must keep being caught, within a 10-point tolerance. A known gap is a strict expected failure, so the suite fails when a change closes it and prompts a refresh.

Model quality

1. **Split by time.** The model trains on the first 75% of the timeline and is tested on the last 25%. This copies real use, where the future is predicted from the past, and prevents leakage.
2. **Metrics suited to rare fraud.** Precision, recall, F1 and PR-AUC are used, not accuracy. Fraud is about 1.5% of rows, so a model that never flags anything would score 98.5% accuracy.

3. **Tuning without peeking.** Settings were chosen inside the training period. The alert threshold comes from out-of-fold predictions, so the test set never influences it.
4. **A baseline.** A plain forest without customer history features gives the reference point for judging improvement.
5. **Adversarial testing.** The red team (section 7) measures performance against fraud designed to evade the model.

Checked by hand: the dashboard in a browser and the example API client script.

Not yet done: load testing, a security review of the API, and validation on real transaction data.

6. Results and accuracy

The tuned model improved on the baseline on every measure: recall rose from 61.7% to 79.6% and precision from 77.6% to 88.4%. The figures below come from the held out test period of 25,703 transactions.

Metric	Baseline	Tuned model	What it means
Precision	77.6%	88.4%	Of the transactions flagged as fraud, this share really were fraud
Recall	61.7%	79.6%	Of all real fraud, this share was caught
F1	0.688	0.838	One score that balances precision and recall
PR-AUC	0.730	0.889	How well fraud is ranked above normal activity across every possible cut-off (1.0 is perfect)
ROC-AUC	0.938	0.981	A looser ranking measure that looks flattering when fraud is rare

In plain numbers, the test period held 392 fraudulent transactions. The model caught about 312 and missed about 80. It wrongly flagged about 41 normal transactions out of roughly 25,300, a false-alarm rate of about 0.16%. These counts are worked out from the rates above, so treat them as approximate.

Accuracy is deliberately not reported. With about 1.5% fraud, a model that flagged nothing would be 98.5% “accurate” and useless.

What drives the decisions. The five most important features were new devices (about 25%), time since the customer’s last transaction (22%), amount (16%), amount compared with the customer’s usual spend (9%) and merchant category (7%). Importance shows what the model relies on, not what causes fraud.

Speed. Rewriting feature preparation to work on whole tables cut its run time by about 98% (roughly 3 to 4 seconds down to 0.05 seconds for 103,000 transactions) in local tests, with identical output. A single API request took about 15 to 20 milliseconds locally, including feature lookup and scoring. These are single-machine figures, not production capacity.

Phishing model. It scored 100% on its own synthetic links. That only shows the code path works, because the test links follow the same patterns the model learned. It says nothing about real phishing.

7. Red-team (adversarial) evaluation

Against simulated attackers that imitate real customers, the deployed model caught only about 51% of held out attacks and let about 54% of the attempted money through, although it caught obvious fraud every time.

The red team ran 12 scenarios against 60 customers each. Five were classic patterns, such as a fast smash and grab and card testing with tiny charges. Three were deliberate disguises: low and slow, daytime camouflage and a full mimic of the customer's habits. Four more were evolved by a search that mutated strategies over six generations, seeing only the model's scores, and kept those that stole the most without being flagged. Over those generations the best attacker's average haul rose from about $700 to about $3,800 per attempt. Every attack ran against a copy of the customer's profile, so no live data was touched, and an account froze at its first flagged transaction.

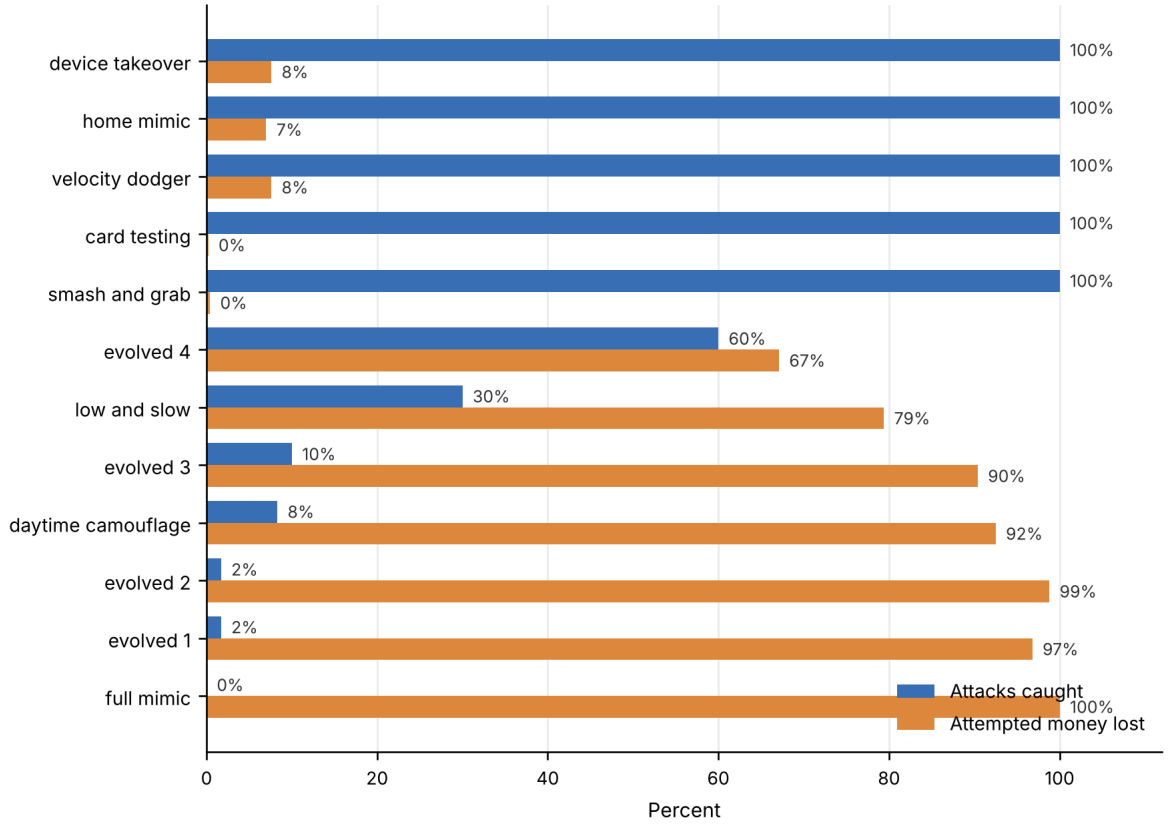


Figure 2. Obvious attacks are caught every time. Disguised and evolved attacks mostly get through.

Blind spots. A small decision tree was fitted to the transactions that slipped through and turned into readable rules:

- **Long gap, known device, web or mobile channel:** about 1,630 transactions missed, and about 73% of normal traffic falls in the same region. This gap is ambiguous: closing it would mean flagging genuine customers.
- **Long gap, new device, amount near the customer's norm:** about 118 transactions missed, and only about 1% of normal traffic matches. This gap is closable at little cost.

Candidate fixes were each retrained and compared on customers and attack types the training never saw, at the same false alarm rate as the deployed model. Differences carry a 95% bootstrap interval, and a fix is rejected if it raises false positives or loses more than 2 points of real fraud recall.

Candidate fix	Verdict	Detection change (95% interval)	Reason
Lower the alert threshold	Rejected	+13.2 pts (+11.4 to +15.2)	False alarms rose from 0.17% to 0.65%
Train on red-team attacks	Rejected	+30.7 pts (+28.0 to +33.5)	Recall on real fraud fell from 79.6% to 76.0%, and unseen attack types got 8.5 points worse
Add customer novelty features	Inconclusive	+0.3 pts (−0.4 to +1.1)	No measurable gain
Novelty features plus attack training	Rejected	+31.2 pts (+28.4 to +34.0)	Recall on real fraud fell from 79.6% to 75.3%

These figures come from one run with a fixed random seed. Running again on another machine can shift them slightly.

No fix was accepted, which is itself a useful result: the quick fixes either buy detection with false alarms or with recall on ordinary fraud. Fraud that copies a customer's habits probably needs new signals, such as device fingerprints or payee history, not more tuning. The attackers here are simulated and limited to the features the model uses, so these rates are not real world fraud rates.

8. Limitations, risks and responsible use

The biggest limitation is that every number in this report comes from simulated data, so none of it predicts how the system would perform on real bank traffic.

- **Simulated data.** The fraud patterns were written by the same person who built the model's features, which favours the model. Real fraud is messier and changes over time.
- **One test split.** The headline figures come from a single time ordered split without confidence intervals, so small differences are not proven.
- **The features set the ceiling.** Fraud that mimics the customer's own habits is mostly invisible (section 7).
- **Drift.** Fraudsters adapt, so a model trained once will decay. A real deployment needs monitoring and scheduled retraining.

- **Prototype shortcuts.** Customer history lives in one process's memory and is lost on restart. One lock guards all updates. A repeated event would be counted twice. The default API key is a development placeholder. The free hosting tier sleeps when idle. None of this is acceptable for production.
- **Phishing model.** It was trained and tested on synthetic links, so its 100% score has no real-world meaning.

Responsible use. A fraud score should support a human decision, not replace it. Signals such as country can flag some customers more often than others, so a real deployment would need a fairness review, a way for customers to dispute a block, and handling of card and personal data that follows the applicable rules (for example PCI DSS and privacy law), which are outside the scope of this prototype.

9. Roadmap

The next work should first make the results credible, then make the system production ready, and only then make it adapt on its own.

1. **Make the results credible.** Retrain on a public real world dataset such as PaySim, add confidence intervals, tune gradient boosting against the Random Forest, and add load and security testing.
2. **Make it production-ready.** Build the AWS design in section 3 in stages: shared customer history in Redis, durable alerts in DynamoDB, versioned models in S3, and a release pipeline that blocks any model that regresses on the red-team corpus.
3. **Make it adapt.** Monitor score drift, feed analyst decisions back as new labels, release new models through shadow mode and a 5% canary, and add signals aimed at the blind spots found by the red team.